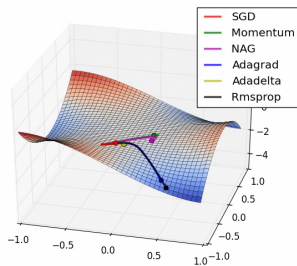
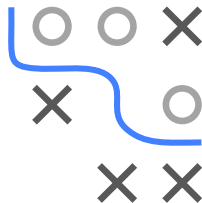


Optimization in Machine Learning

First order methods

Adam and friends

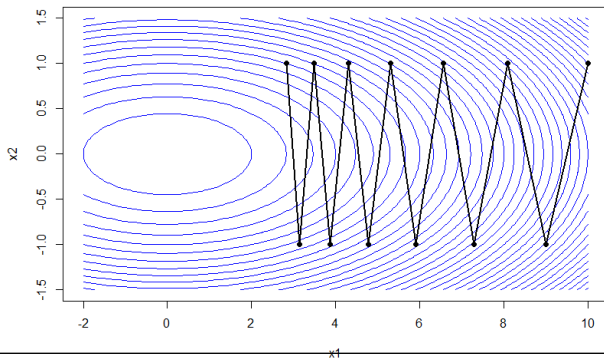
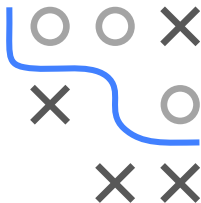


Learning goals

- Understand the idea of adaptive step size methods
- Understand the adaptive step size methods AdaGrad, RMSProp, and Adam and their differences

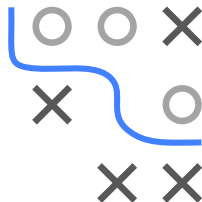
ADAPTIVE STEP SIZES

- Besides descent direction, step size is the other important control parameter in first order methods with strong influence on performance
- Motivated by limitations of global step sizes in poorly-conditioned problems, it is natural to use different step size for each dimension individually and automatically adapt sizes over time



ADAGRAD

- AdaGrad adapts step sizes by scaling them inversely proportional to square root of the sum of the past squared derivatives
 - Inputs with large derivatives get smaller step sizes
 - Inputs with small derivatives get larger step sizes
- This effectively changes the direction of the update (and not just the magnitude) by scaling the gradient element-wise

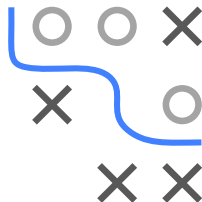


Algorithm AdaGrad

- 1: **require** Initial parameter θ , Global step size α , Small constant β for numerical stability (e.g. 10^{-7})
 - 2: **Initialize** Gradient accumulation variable $\mathbf{r} = \mathbf{0}$
 - 3: **while** stopping criterion not met **do**
 - 4: Sample minibatch of m examples from the training set $\{\tilde{\mathbf{x}}^{(1)}, \dots, \tilde{\mathbf{x}}^{(m)}\}$
 - 5: Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(y^{(i)}, f(\tilde{\mathbf{x}}^{(i)} | \theta))$
 - 6: Accumulate squared gradient $\mathbf{r} \leftarrow \mathbf{r} + \hat{\mathbf{g}} \odot \hat{\mathbf{g}}$ (where $\odot$: element-wise product)
 - 7: Compute update: $\nabla \theta = -\frac{\alpha}{\beta + \sqrt{\mathbf{r}}} \odot \hat{\mathbf{g}}$
 - 8: Apply update: $\theta \leftarrow \theta + \nabla \theta$
 - 9: **end while**
-

RMSPROP

- AdaGrad's accumulation of squared gradients via summation can result in radically diminishing step sizes
- RMSProp is a modification of AdaGrad that resolves this by using an exponentially weighted moving average of past squared gradients
- Theoretically, RMSProp leads to performance gains in non-convex scenarios. Empirically, it is a very effective optimization algorithm routinely employed by DL practitioners.

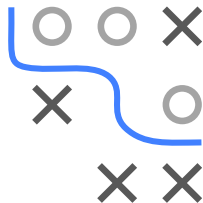


Algorithm RMSProp

- 1: **require** Initial parameter θ , Global step size α , Small constant β for numerical stability (e.g. 10^{-7}), decay rate $\rho \in [0, 1)$
 - 2: **Initialize** gradient accumulation variable $\mathbf{r} = \mathbf{0}$
 - 3: **while** stopping criterion not met **do**
 - 4: Sample minibatch of m examples from the training set $\{\tilde{\mathbf{x}}^{(1)}, \dots, \tilde{\mathbf{x}}^{(m)}\}$
 - 5: Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(y^{(i)}, f(\tilde{\mathbf{x}}^{(i)} | \theta))$
 - 6: Accumulate squared gradient $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \hat{\mathbf{g}} \odot \hat{\mathbf{g}}$
 - 7: Compute update: $\nabla \theta = -\frac{\alpha}{\beta + \sqrt{\mathbf{r}}} \odot \hat{\mathbf{g}}$
 - 8: Apply update: $\theta \leftarrow \theta + \nabla \theta$
 - 9: **end while**
-

ADAM

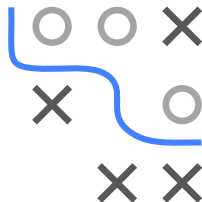
- Adaptive Moment Estimation (ADAM) uses the 1st and 2nd moments of gradients
 - 1st moment: Exponentially decaying average of past gradients as momentum term
 - 2nd moment: Exponentially decaying average of past squared gradients to adapt step sizes like RMSProp
- Can thus be seen as combination of RMSProp and momentum



ADAM

Algorithm Adam

- 1: **require** Initial parameters θ , Global step size α (suggested default: 0.001), Small constant β (suggested default 10^{-8}), Exponential decay rates for moment estimates ρ_1 and ρ_2 in $[0, 1)$ (suggested defaults: 0.9 and 0.999 respectively)
 - 2: **Initialize** time step $t = 0$
 - 3: **Initialize** 1st and 2nd moment variables $\mathbf{s}^{[0]} = 0, \mathbf{r}^{[0]} = 0$
 - 4: **while** stopping criterion not met **do**
 - 5: $t \leftarrow t + 1$
 - 6: Sample a minibatch of m examples from the training set $\{\tilde{\mathbf{x}}^{(1)}, \dots, \tilde{\mathbf{x}}^{(m)}\}$
 - 7: Compute gradient estimate: $\hat{\mathbf{g}}^{[t]} \leftarrow \frac{1}{m} \nabla_{\theta} \sum_i L(y^{(i)}, f(\tilde{\mathbf{x}}^{(i)} | \theta))$
 - 8: Update biased first moment estimate: $\mathbf{s}^{[t]} \leftarrow \rho_1 \mathbf{s}^{[t-1]} + (1 - \rho_1) \hat{\mathbf{g}}^{[t]}$
 - 9: Update biased second moment estimate: $\mathbf{r}^{[t]} \leftarrow \rho_2 \mathbf{r}^{[t-1]} + (1 - \rho_2) \hat{\mathbf{g}}^{[t]} \odot \hat{\mathbf{g}}^{[t]}$
 - 10: Correct bias in first moment: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}^{[t]}}{1 - \rho_1^t}$
 - 11: Correct bias in second moment: $\hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}^{[t]}}{1 - \rho_2^t}$
 - 12: Compute update: $\nabla \theta = -\frac{\alpha}{\beta + \sqrt{\hat{\mathbf{r}}}} \odot \hat{\mathbf{s}}$
 - 13: Apply update: $\theta \leftarrow \theta + \nabla \theta$
 - 14: **end while**
-



ADAM: BIAS IN MOMENT ESTIMATES

- Initializes moment variables $\mathbf{s}$ and $\mathbf{r}$ with zero $\Rightarrow$ Bias towards zero

$$\mathbb{E}[\mathbf{s}^{[t]}] \neq \mathbb{E}[\hat{\mathbf{g}}^{[t]}] \quad \text{and} \quad \mathbb{E}[\mathbf{r}^{[t]}] \neq \mathbb{E}[\hat{\mathbf{g}}^{[t]} \odot \hat{\mathbf{g}}^{[t]}]$$

($\mathbb{E}$ calculated over minibatches)

- Indeed: Unrolling $\mathbf{s}^{[t]}$ yields

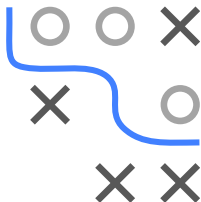
$$\mathbf{s}^{[0]} = 0$$

$$\mathbf{s}^{[1]} = \rho_1 \mathbf{s}^{[0]} + (1 - \rho_1) \hat{\mathbf{g}}^{[1]} = (1 - \rho_1) \hat{\mathbf{g}}^{[1]}$$

$$\mathbf{s}^{[2]} = \rho_1 \mathbf{s}^{[1]} + (1 - \rho_1) \hat{\mathbf{g}}^{[2]} = \rho_1 (1 - \rho_1) \hat{\mathbf{g}}^{[1]} + (1 - \rho_1) \hat{\mathbf{g}}^{[2]}$$

$$\mathbf{s}^{[3]} = \rho_1 \mathbf{s}^{[2]} + (1 - \rho_1) \hat{\mathbf{g}}^{[3]} = \rho_1^2 (1 - \rho_1) \hat{\mathbf{g}}^{[1]} + \rho_1 (1 - \rho_1) \hat{\mathbf{g}}^{[2]} + (1 - \rho_1) \hat{\mathbf{g}}^{[3]}$$

- Therefore: $\mathbf{s}^{[t]} = (1 - \rho_1) \sum_{i=1}^t \rho_1^{t-i} \hat{\mathbf{g}}^{[i]}$
- Note:** Contributions of past $\hat{\mathbf{g}}^{[i]}$ decrease rapidly



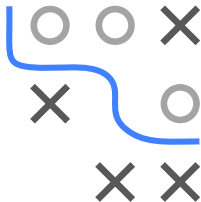
ADAM: BIAS CORRECTION

- We continue with

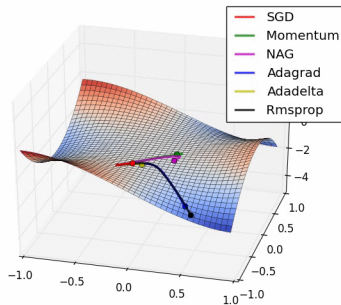
$$\begin{aligned}\mathbb{E}[\mathbf{s}^{[t]}] &= \mathbb{E}[(1 - \rho_1) \sum_{i=1}^t \rho_1^{t-i} \hat{\mathbf{g}}^{[i]}] \\ &= \mathbb{E}[\hat{\mathbf{g}}^{[t]}](1 - \rho_1) \sum_{i=1}^t \rho_1^{t-i} + \zeta \\ &= \mathbb{E}[\hat{\mathbf{g}}^{[t]}](1 - \rho_1^t) + \zeta,\end{aligned}$$

where we approximated $\hat{\mathbf{g}}^{[i]}$ by $\hat{\mathbf{g}}^{[t]}$. The resulting error is put in ζ and is kept small due to the exponential weights of past gradients

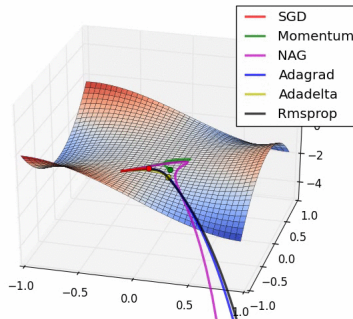
- Therefore: $\mathbf{s}^{[t]}$ is a biased estimator of $\hat{\mathbf{g}}^{[t]}$
- But bias vanishes for $t \rightarrow \infty$ ($\rho_1^t \rightarrow 0$)
- Ignoring ζ , we correct for the bias by $\hat{\mathbf{s}}^{[t]} = \frac{\mathbf{s}^{[t]}}{(1 - \rho_1^t)}$
- Analogously: $\hat{\mathbf{r}}^{[t]} = \frac{\mathbf{r}^{[t]}}{(1 - \rho_2^t)}$



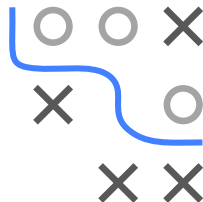
COMPARISON OF OPTIMIZERS: ANIMATION



► [Click for source](#)



► [Click for source](#)



Comparison of SGD variants near saddle point

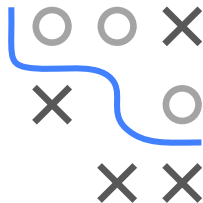
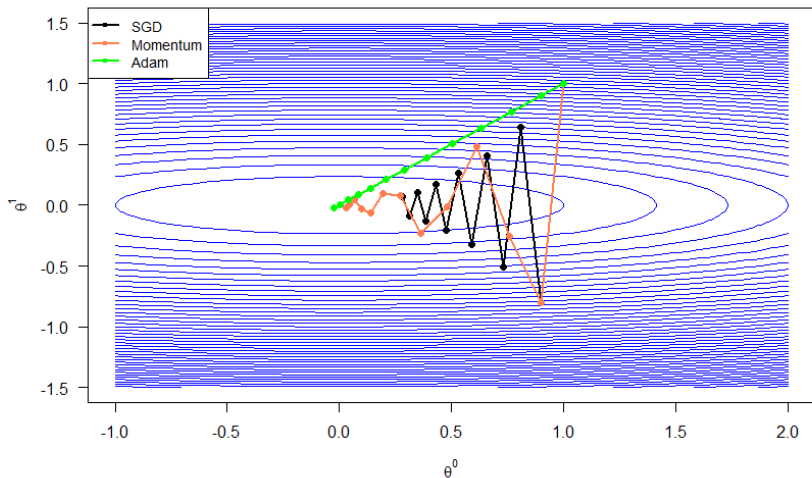
Left: After start. **Right:** Later

All methods accelerate compared to vanilla SGD

Best is RMSProp, then AdaGrad (Adam is missing here)

Credits: Dettmers (2015) and Radford

COMPARISON ON QUADRATIC FORM



SGD vs SGD with Momentum vs Adam on a quadratic form